The Anatomy of the

.claude/ folder

A comprehensive guide to structuring, configuring, and extending Claude Code projects

Why Your .claude/ Folder Is Critical

Every token counts - well-structured configuration saves money and improves results.

Token Efficiency

Models degrade after ~128K tokens. Skills use progressive disclosure to load only what's needed, keeping your context lean.

Consistent Quality

Rules and skills embed best practices into every session. Claude doesn't start from scratch - it starts informed.

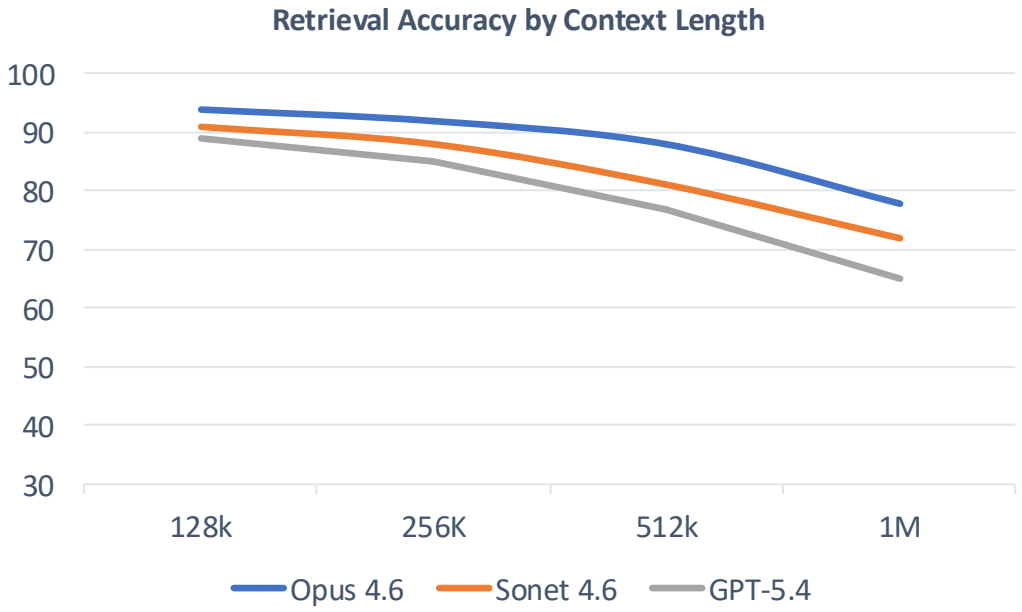
Cost Savings

Every unnecessary token in context is wasted spend. Efficient structure = lower bills.

Bottom Line: A well-organized .claude/ folder is the difference between a $0.05 task and a $0.50 task - with better results.

Model Performance Degrades With Context Length

Research shows accuracy drops significantly as token count grows - the "Lost in the Middle" effect.



Instruction following degrades fastest - complex tasks fail before retrieval

Cost scales linearly but quality drops exponentially

What is .claude/?

The .claude/ directory is the configuration backbone of every Claude Code project. It tells Claude who it is, what it can do, and how to behave

Context

Gives Claude deep knowledge about your project, codebase, and team conventions.

Configuration

Controls permissions, tool access, MCP servers, and environment variables.

Automation

Enables custom skills, agents, and hooks that extend Claude's capabilities.

Two-Level Architecture

Claude uses a layered configuration system. User-level applies everywhere, project-level is scoped to each repo.

~/.claude/

User-Level (Global)

CLAUDE.md
— Global instructions

settings.json
— User preferences

rules/
— Personal modular rules

skills/
— Personal automation skills

.claude/

Project-Level (Repo Root)

settings.json
— Shared team config

settings.local.json
— Personal overrides

rules/
— Team modular rules

skills/
— Team shared workflows

agents/
— Subagent definitions

Priority: settings.local.json > .claude/settings.json > ~/.claude/settings.json

Complete Folder Map

```
.claude/
├── CLAUDE.md
├── settings.json
├── settings.local.json
├── rules/
│   ├── testing.md
│   └── react-patterns.md
├── skills/
│   ├── my-skill/
│   │   ├── SKILL.md
│   │   └── scripts/
│   └── deploy/
│       └── SKILL.md
├── agents/
│   └── code-reviewer.md
└── .gitignore
```

`CLAUDE.md`
Project context & instructions

`settings.json`
Permissions & configuration

`rules/`
Modular instruction files

`skills/`
Auto-invoked workflow packages

`agents/`
Specialized subagent configs

Core Configuration

CLAUDE.md, settings.json, and rules/

CLAUDE.md

The project's instruction file - Claude reads it at the start of every session.

What to Include

- Project purpose — the WHY, WHAT, HOW
- Build & test commands (exact CLI)
- Code style & architecture decisions
- Preferred libraries and patterns
- Known gotchas & warnings

Hierarchy

`~/.claude/CLAUDE.md`
Global (all projects)

`/CLAUDE.md`
Project root

`~/.claude/CLAUDE.local.md`
Personal (gitignored)

Best Practice: Keep it under 500 words. Commit to git for team sharing.

settings.json

Controls what Claude is allowed to do - tool access, permissions, MCP servers, and hooks.

User-Level

`~/.claude/settings.json`

Applies across all projects. Your personal defaults.

Project-Level

`.claude/settings.json`

Shared team config. Checked into git.

Local Override

`.claude/settings.local.json`

Personal project overrides. Auto-gitignored.

```
{
  "permissions": {
    "allow": ["Read", "Write", "Bash"],
    "deny": ["WebFetch"]
  },
  "env": { "NODE_ENV": "development" },
  "mcpServers": { ... },
  "hooks": { ... }
}
```

rules/

Modular instruction files - break up CLAUDE.md into organized, topic-specific rules.

```
.claude/rules/
├── testing.md
├── api-design.md
├── security.md
├── react-patterns.md
├── styling.md
├── database.md
└── authentication.md
```

Key Benefits

Auto-discovered

All .md files load automatically

No config needed

Just create files and they work

Topic-focused

One concern per file

Path-scoped

Rules can target specific directories

When to use: When CLAUDE.md exceeds ~200 lines, split domain-specific instructions into rules/. Use descriptive filenames (testing.md, not rules1.md). Organize by feature area with subdirectories.

Skills, Agents & Automation

The power trio that makes Claude truly yours

skills/

Auto-invoked workflow packages - teach Claude repeatable processes with YAML frontmatter.

Skill Folder Structure

```
my-skill/
├── SKILL.md        # Required
├── template.md     # Optional
├── reference.md    # Optional
├── examples/
└── scripts/
    ├── validate.sh
    └── helper.py
```

SKILL.md Format

```
---
name: my-skill
description: What it does.
  Use when user asks to...
allowed-tools: Read Grep Glob
---

# Instructions
Step-by-step guidance...
```

Progressive Disclosure

1 YAML Frontmatter

Always loaded - decides when to invoke

2 SKILL.md Body

Loaded when skill activates

3 Linked Files

Loaded on demand by Claude

Using Skills in the Terminal

Manual Invocation

```
$ /deploy production

$ /review-code src/auth/login.ts

$ /migrate-component SearchBar React Vue
```

Automatic Invocation

Claude reads skill descriptions at startup. When your request matches a skill's description, Claude loads and follows that skill automatically.

Only ~100 tokens per skill for discovery!

Key Frontmatter Controls

Field	Purpose	Example
`name`	Becomes /slash-command	`name: deploy`
`description`	Triggers auto-invocation	`description: Deploy to prod...`
`allowed-tools`	Tools without permission ask	`allowed-tools: Read Bash`
`disable-model-invocation`	Manual only (no auto)	`disable-model-invocation: true`
`context: fork`	Run in isolated subagent	`context: fork`

Skills + MCP: Better Together

MCP provides the tools. Skills provide the recipes. Together they enable powerful, consistent workflows.

MCP (Connectivity)	Skills (Knowledge)
Connects Claude to services (Jira, Jenkins, PowerPoint, etc.)	Teaches Claude how to use those services effectively
Real-time data access and tool invocation	Captures workflows and best practices
What Claude can do	How Claude should do it

Without Skills

- Users connect MCP but don’t know what to do
- Each conversation starts from scratch
- Inconsistent results every time

With Skills

- Pre-built workflows activate automatically
- Consistent, reliable tool usage
- Best practices embedded in every interaction

agents/

Specialized subagents - delegate specific types of tasks with custom prompts and restricted tools.

```
code-reviewer.md

name: code-reviewer
description: Review code for
  quality and security
tools:
  - Read
  - Grep
  - Glob

# Code Review Agent
You are a senior code reviewer.
Check for security issues,
performance patterns, tests.
```

Common Agent Patterns

Code Reviewer
Read, Grep, Glob

Test Runner
Read, Bash, Write

Doc Writer
Read, Write, Glob

Research Agent
Read, WebSearch, Grep

Security Auditor
Read, Grep, Bash

Key principle: Each agent runs in its own context with independent permissions. Grant only the tools the agent actually needs.

Hooks

Event-driven automation - run commands at specific points in Claude's lifecycle.

`SessionStart`

Initialize at session start

`UserPromptSubmit`

When user sends a prompt

`PostToolUse`

After any tool executes

`PreToolUse`

Before a tool runs

`PostCommandExec`

After a command completes

`ToolPermissionReq`

When approval is needed

Common Use Cases

Auto-format

Run Prettier after every edit

Inject context

Load tickets at session start

Enforce lint

Block commits that fail checks

MCP Servers

Extend Claude's capabilities with external tools and data sources via Model Context Protocol.

mcpServers in settings.json

```
"mcpServers": {
  "github": {
    "type": "stdio",
    "command": "gh-mcp",
    "args": ["-r", "github/repo"]
  },
  "notion": {
    "type": "sse",
    "url": "http://localhost:3000",
    "headers": {
      "authorization": "Bearer $TOKEN"
    }
  }
}
```

Transport Types

`stdio`
Local process communication

`http`
Remote HTTP endpoints

`sse`
WebSocket connections

Management

```
claude mcp add <name> <url>
/mcp enable <server>
```

The Virtuous Cycle

Good skills + agents + CLAUDE.md = less tokens spent = better model performance = lower costs.

Well-Structured .claude/ Folder

Skills, rules, agents all configured

Less Context Needed

Progressive disclosure loads only what's needed

Fewer Tokens Spent

Stay in the 30–50K sweet spot

Peak Model Performance

88–98% accuracy instead of 55–70%

Best Practices Cheat Sheet

CLAUDE.md

- Keep under 500 words
- Use /init to bootstrap
- Commit to git

Settings

- Secrets in local.json only
- Use $ENV_VAR syntax
- Run /status to debug

Rules

- One topic per file
- Descriptive filenames
- Organize by domain

Skills

- Kebab-case folder names
- SKILL.md is case-sensitive
- Use progressive disclosure
- Keep body < 500 lines

Agents

- Restrict tool access
- Clear role descriptions
- Specialized > general

MCP & Hooks

- Run tools in settings.json
- Tokens in env variables
- Keep hooks fast

Quick Reference

Entity	Location	Purpose
`CLAUDE.md`	`.claude/ or project root`	Project instructions & context
`settings.json`	`.claude/settings.json`	Permissions, MCP, hooks config
`settings.local.json`	`.claude/settings.local.json`	Personal overrides, API keys
`rules/`	`.claude/rules/`	Modular instruction files
`skills/`	`.claude/skills/`	Auto-invoked workflow packages
`agents/`	`.claude/agents/`	Subagent definitions

Essentials

Browsing the Marketplace

Use **/plugin** in the Claude Code terminal to browse available plugin marketplaces. Navigate to the marketplace to discover and install skills that extend Claude's capabilities.

Must-Have Skills

superpowers - Unlocks advanced agentic workflows, giving Claude Code enhanced abilities to handle complex, multi-step tasks autonomously.

skill-creator - Lets you build, test, and refine your own custom skills. Instead of waiting for someone else to publish what you need, you can create purpose-built skills tailored to your exact workflow

next ○ **21 APR 17:00** ○ **Workshop**
The Anatomy of the .claude/ folder